

Towards Distributed Validators on leanConsensus

DV Labs

February 2026

1 Introduction

Ethereum’s transition to proof-of-stake introduces a critical dependency on *validator signatures*: every attestation and block proposal must be signed by an authorised validator. The consensus layer currently relies on BLS12-381 for its native support for signatures aggregation. However, on the quantum horizon, BLS12-381 signatures will be broken by Shor’s algorithm running on a cryptographically-relevant quantum computer (CRQC). To address this issue, the Beacon chain is being redesigned as leanConsensus. Its core component is a hash-based zkVM aggregation system designed to address both the *post-quantum* and the *aggregation efficiency* requirements simultaneously. Its core contributions are:

- (i) A Cairo-inspired zkVM (leanVM) whose execution traces are proven by a WHIR-based SNARK.
- (ii) An aggregation circuit defined in a domain-specific language (DSL) that can aggregate arbitrarily many XMSS signatures in a single succinct proof via a hierarchical tree of sub-proofs.

This document provides an overview of leanConsensus, with particular focus on how it can be extended to support threshold signatures, and what open engineering challenges remain for distributed validator (DV) solutions.

2 Architecture of leanConsensus

2.1 Cryptographic Primitives

Base field. All arithmetic is performed over the *KoalaBear* prime field $\mathbb{F}_p$ with $p = 2^{31} - 2^{24} + 1$ (a 31-bit prime with 2-adicity 24). The extension used for polynomial commitments is $\mathbb{F}_{p^5}$, a degree-5 extension, giving a security level of 123 bits under the Johnson bound and 128 bits conjectured under the capacity bound. Every field element is 31 bits; an 8-element “digest” (the unit of hash output used throughout) is $31 \times 8 = 248$ bits.

Hash function. Poseidon2 with a 16-element ($16 \times 31 = 496$ -bit) state is used everywhere using the compression mode as it builds all Merkle trees in the system. Poseidon2 is algebraically friendly so that it integrates naturally into the AIR/Logup constraint system.

WOTS⁺ one-time signatures. WOTS⁺ is the leaf-level signing primitive. A WOTS⁺ private key $\mathbf{sk}_i = (s_1, \dots, s_V) \in \mathbb{F}_p^{V \cdot 8}$ generates a public key by iterating Poseidon2 along each chain. To sign, the hash encoding of the message is decomposed into per-chain lengths $e_1, \dots, e_V$ with $\sum e_i = T$, and each chain tip is revealed at position e_i from the start.

XMSS signatures. An XMSS key pair covers a lifetime of 2^n Ethereum *slots* where n is fixed at key generation time. The public key is a single digest $r_{\text{root}} \in \mathbb{F}_p^8$ representing the root of a Merkle tree of depth n over all WOTS⁺ leaf hashes. A signature at slot s consists of:

- The WOTS⁺ randomness (7 field elements).
- V chain tips ($V \times 8$ field elements).
- A Merkle authentication path of depth n ($n \times 8$ field elements).

2.2 The leanVM Execution Model

leanVM is a register-free, memory-based virtual machine inspired by Cairo [GPR21]. Its memory model is *write-once*: each address can be written at most once; a second write to the same address triggers a constraint violation. This property is central to correctness proofs. The instruction set includes:

- **Arithmetic:** ADD, MUL over $\mathbb{F}_p$ and $\mathbb{F}_{p^5}$.
- **Memory:** load from and write to the write-once memory.
- **Control flow:** unconditional and conditional jumps; the program counter advances monotonically, enabling efficient AIR constraints.
- **Precompiles:** poseidon16 and dot-product accelerators.

The execution trace is a table with three sub-tables: the *Execution table* (instruction columns + runtime witness columns), the *DotProduct table*, and the *Poseidon16 table*. The three tables are connected via the *Logup bus*: each precompile call inserts a (fingerprinted) row into the corresponding table, and the Logup argument [Hab22] proves that the multisets match.

2.3 Proving Pipeline

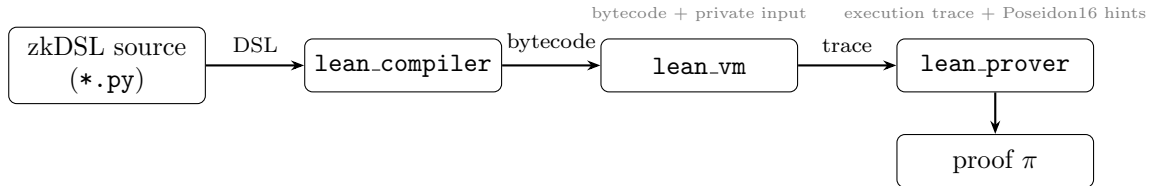


Figure 1: Proving pipeline

SuperSpartan. The main proof system is a variant of Spartan [Set20] operating over extension field multilinear polynomials. The execution table’s correctness is expressed as a sum over the Boolean hypercube of a degree-3 polynomial; a single sumcheck invocation reduces it to a point evaluation.

WHIR. Polynomial commitment proofs are generated with WHIR [ACFY25], a Reed–Solomon proximity protocol with:

- Initial folding factor 7, subsequent factor 5.
- 18 grinding bits (proof-of-work).
- Code rate $\rho \in \{1/2, 1/4\}$ selectable per node.

Logup. The lookup argument connecting the three tables follows the *logarithmic derivative* technique [Hab22]. The bus value is a linear combination of the table row, fingerprinted with Fiat-Shamir challenges.

2.4 Aggregation Tree Topology

A single `leanConsensus` node aggregates at most one “level” of the tree. Each node’s circuit (`main.py`) handles two source types:

1. **Raw XMSS** — individual validators signing directly.
2. **Recursive children** — previously-proven aggregation nodes whose proofs are verified inside this node’s circuit.

The public state of a node is an `AggregatedSigs` structure:

- A list of public keys ($8 \times n$ field elements).
- A proof π (a few hundred KiB, constant regardless of n).
- A *bytecode claim* (an evaluation of the bytecode multilinear at a random point), used for recursive verification.

The global public key set is a deduplicated, sorted union of all raw and child public keys. Its *Poseidon-slice-hash* (i.e., the running hash obtained by sequentially hashing consecutive pairs of pubkey digests) is committed in the proof’s public input.

Bytecode claim reduction. When a node has n_r recursive children, it must prove that each child was proved with the same bytecode. The $2n_r$ bytecode claims (one per child, times two for the two halves of the recursion check) are collapsed into a single evaluation via a random linear combination and a sumcheck over the bytecode multilinear polynomial. The resulting evaluation point is propagated to the parent node.

3 Threshold Hash-Based Signatures

3.1 Motivation: Distributed Validators

In Ethereum’s Distributed Validator (DV) model [Fou23], a single logical validator is operated by a cluster of n independent nodes. This architecture merges threshold cryptography with Byzantine Fault Tolerant (BFT) consensus to eliminate single points of failure, optimize uptime, and prevent equivocation.

DKG and threshold signatures. Rather than relying on a trusted dealer to generate and distribute the private signing key, which would represent a central point of failure, the cluster performs a Distributed Key Generation (DKG) protocol. This allows the n operators to collectively compute a public verification key while each operator strictly learns only their secret share. In the pre-quantum BLS-based DV setting, the private signing key is split amongst multiple parties using a linear secret sharing scheme. To produce a threshold BLS12-381 signature, a signer signs the agreed-upon data with its key share. Because of the homomorphic properties unique to pairing-based schemes, k partial BLS signatures can be linearly combined into a single valid BLS signature that is indistinguishable from a standard, non-threshold signature.

Intra-cluster consensus and BFT. While threshold cryptography dictates *how* a signature is constructed, it does not dictate *what* is signed. Before any partial signatures are generated, the n nodes must run a BFT consensus protocol (e.g., an Istanbul BFT variant) to deterministically agree on the exact Beacon Chain duty (block or attestation data) they will execute. By enforcing consensus prior to signing, the DV guarantees that no honest operator will sign conflicting data, effectively eliminating the risk of accidental slashing.

System guarantees and threshold parameters. To tolerate malicious (Byzantine) actors within the cluster, the system parameters are typically bound by the BFT threshold $n \geq 3f + 1$, where f is the maximum number of Byzantine nodes. Consequently, the cryptographic signing threshold k is usually set to $k \geq 2f + 1$ (often represented as $k = \lceil \frac{2n}{3} \rceil$).

Impactical indistinguishable threshold hash-based signatures. Because the BLS homomorphic property abovementioned is unique to pairing-based schemes and does not transfer to hash-based signatures, under a post-quantum signature scheme like XMSS, threshold sharing via polynomial interpolation is not natively supported anymore. Therefore, supporting threshold signatures requires ad-hoc mechanisms. In order to be transparent at the protocol level, a key requirement for threshold signatures is to be indistinguishable from non-threshold ones to be compliant with the network specification. Amongst the different possible solutions, leveraging multi-party computation (MPC) has been shown to fall short in terms of performance [?] due to high number of hash calls to be computed in a distributed manner. A method based on Boolean sharing has also been proposed by Kelsey, Lang and Lucks in [KLL25] but it requires a trusted party and a large common reference string (around 1 GiB per key). Because both options are not satisfactory either in terms of performance or memory consumption, we hereafter consider other approaches that affect the signature format (i.e. threshold signatures are distinguishable from non-threshold ones).

3.2 Proof-Based Threshold Signatures

Leveraging proof systems to construct threshold hash-based signatures has already been explored by Khaburzaniya, Chalkias, Lewi and Malvai [KCLM22]. In this approach, an aggregator combines individual signatures over the same message into a zero-knowledge proof that serves as the actual threshold signature. More precisely, given a k -out-of- n setting, the aggregator can generate a proof attesting that it verified k distinct signatures over the same message and that signers are part of the quorum. A valuable feature of this approach is its non-interactiveness: the aggregator only needs to collect individual signatures in order to compute the proof, without any additional communication. On the other hand, it inflates the signature size by over an order of magnitude, making the resulting threshold signatures incompatible with standard signature formats. Furthermore, proof-based threshold signatures are ill-suited for the aggregation mechanisms central to leanVM. Integrating them introduces a new recursion layer for every threshold signature processed. Because each recursion layer takes an `AggregatedSig` alongside raw XMSS signatures as input, verifying the `AggregatedSig` within the circuit quickly becomes a severe performance bottleneck.

3.3 Dummy Threshold Signatures

Another approach would be to use *dummy* threshold signatures that simply consist of at least k individual signatures from authorized signers within a *threshold group* as defined below.

Definition 3.1 (Threshold Group). *A threshold group $G = (k, n, \mathcal{T})$ consists of:*

- k (threshold): *the minimum number of signers required.*
- n (group size): *the total number of members.*
- $\mathcal{T}$: *a Merkle tree of depth $d = \lceil \log_2 n \rceil$, whose 2^d leaves are the n individual public keys $r_1, \dots, r_n$ padded with zero digests to the nearest power of two.*

The joint public key of G is its hypertree root $r_G \in \mathbb{F}_p^8$.

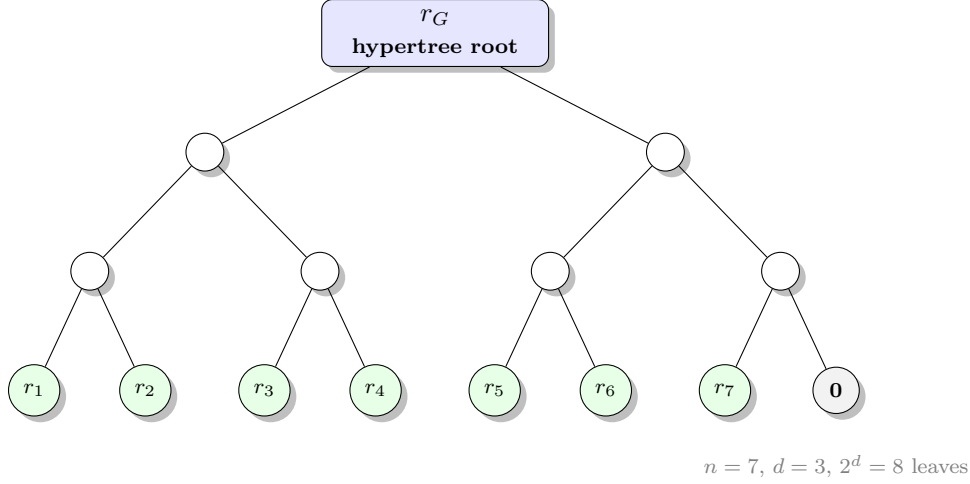


Figure 2: Hypertree for a x -out-of-7 threshold group ($n = 7$, depth $d = 3$). Seven XMSS public keys $r_1, \dots, r_7$ are placed as leaves; the eighth leaf is zero-padded. Each internal node is a Poseidon2₁₆ compression of its two children. The hypertree root r_G is the joint public key appearing in the global `pub_keys` list.

While Definition 3.1 is generic regardless of the k and n parameters, we hereafter assume $n \leq 8$ as distributed validators rarely exceed 8 participants. Indeed most configurations use either $(k, n) = (3, 4)$ or $(5, 7)$. A threshold signature then consists of a set of signer indices, individual signatures and membership proofs as defined hereafter.

Definition 3.2 (Threshold Signature). *Given a group $G = (k, n, \mathcal{T})$ and a message m , a threshold signature is a tuple $\Sigma = (\mathbf{i}, \sigma, \pi)$ where:*

- $\mathbf{i} = (i_1, \dots, i_k)$ is a set of k distinct signer indices, $i_j \in \{1, \dots, n\}$.
- $\sigma = (\sigma_1, \dots, \sigma_k)$ are the k individual XMSS signatures, σ_j being the signature of signer i_j on m .
- $\pi = (\pi_1, \dots, \pi_k)$ are the corresponding Merkle authentication paths, where π_j contains the d sibling hashes required to reconstruct the root r_G from the leaf r_{i_j} .

Threshold signature verification consists of three independent checks per signer:

1. **XMSS validity:** $\text{XMSS.Verify}(r_{i_j}, m, \sigma_j) = \top$.
2. **Hypertree membership:** the Merkle path π_j successfully authenticates the leaf r_{i_j} against the known joint public key r_G .
3. **Distinctness:** no index i_j appears more than once in $\mathbf{i}$.

The combined verification cost is dominated by the k XMSS verifications, each requiring $O(V \cdot L + \log_{\text{lifetime}})$ Poseidon2 calls (WOTS chains + XMSS Merkle path), plus $k \times d$ additional Poseidon2 calls for the hypertree paths.

4 Integration of Threshold Signatures into leanVM

4.1 Overview

The most straightforward way to extend leanVM in order to support dummy threshold signatures is to perform the verification of all k signers in a threshold group *inside the same aggregation*

circuit as the raw XMSS and recursive proof checks. It implies supporting threshold signatures as a new source type in `main.py`, processed between raw XMSS and recursive sources. Unlike proof-based threshold signatures, the dummy threshold verification logic (XMSS verification of each signer plus $d \leq 3$ additional Poseidon2 hashes per signer for the hypertree path) is added as extra rows in the execution trace, not as an additional proof system invocation. The proving cost of extra trace rows is sublinear (only log factors enter through WHIR), so a threshold group with $k = 7$ signers adds roughly the same overhead as 7 extra raw XMSS signatures. Beyond the raw cost argument, it has the advantage to keep a single bytecode program and an homogeneous recursion interface as no new proof type is introduced.

4.2 In-Circuit Threshold Verification

The in-circuit implementation lives in `threshold_aggregate.py` and is compiled into the same bytecode as the rest of `main.py`.

4.2.1 Private Input Layout

Each threshold source block in private memory has the layout:

$$\underbrace{k}_{\text{threshold}} \quad \underbrace{d}_{\text{depth}} \quad \underbrace{j}_{\text{pubkey idx}} \quad \underbrace{\ell_1, \dots, \ell_k}_{\text{leaf indices}} \quad \underbrace{r_{\ell_1}, \dots, r_{\ell_k}}_{\text{signer roots (hints)}} \quad \underbrace{\sigma_1, \dots, \sigma_k}_{\text{XMSS sigs}} \quad \underbrace{\pi_1, \dots, \pi_k}_{\text{hypertree proofs}}$$

where each signer root $r_{\ell_i} \in \mathbb{F}_p^8$ is provided as a private hint (not trusted; it is recomputed and verified in-circuit), and each hypertree proof π_i consists of d sibling digests.

The global private input header was extended from the pre-threshold format:

Layout	Description
[n recursions]	Number of recursive child proofs
[n threshold]	Number of threshold source blocks
[n dup]	Number of duplicate public keys
[ptr_pubkeys]	Pointer to global public key table
[ptr_src_0, ptr_thresh_0..t, ptr_rec_0..n, ptr_bytecode_sumcheck]	Ordered source pointers

4.2.2 Verification Algorithm

For each threshold source block (indexed $t = 0, \dots, n_{\text{threshold}} - 1$), the circuit executes Algorithm 1.

Algorithm 1 `threshold_verify_group` (in-circuit, zkDSL)

Require: k : threshold; d : depth; j : global pubkey index; $\ell_1, \dots, \ell_k$: leaf indices; r_{ℓ_i} : signer root hints; σ_i : XMSS signatures; π_i : hypertree proofs (d siblings each)

- 1: $r_G \leftarrow \text{all_pubkeys}[j]$ ▷ expected hypertree root (public)
 - 2: $\text{seen} \leftarrow \text{Array}(2^d)$ ▷ write-once bitmap for distinctness
 - 3: **for** $i = 0 \rightarrow k - 1$ **do**
 - 4: Assert $\ell_i < 2^d$ ▷ leaf index in bounds
 - 5: $\text{seen}[\ell_i] \leftarrow i + 1$ ▷ write-once: duplicate ℓ_i triggers MemoryAlreadySet
 - 6: XMSS.InCircuitVerify(r_{ℓ_i} , m , σ_i , slot) ▷ writes r_{ℓ_i} to its hint address; memory-write assertion
 - 7: $\hat{r}_G \leftarrow \text{HypertreeMerkleVerify}(r_{\ell_i}, \pi_i, \ell_i, d)$
 - 8: Assert $\hat{r}_G = r_G$ element-wise ▷ explicit equality check
 - 9: **end for**
 - 10: **return** j ▷ global pubkey index used for partition counter
-

4.2.3 Distinctness via Write-Once Memory

The bitmap array `seen[MAX_THRESHOLD_SIGNERS]` uses the write-once memory property of `leanVM` to enforce signer index uniqueness without a sorting circuit or a hash set. On the first write, `seen[ $\ell_i$ ]  $\leftarrow i + 1$` succeeds. On a second write with any other value (since i is strictly increasing), the memory constraint

$$(\text{previous value}) = (\text{new value})$$

is violated, causing an unsatisfiable AIR constraint. This is provably sound under the Logup bus argument: the memory table enforces that every read is consistent with the unique write that created the entry.

4.2.4 Hypertree Merkle Verification (Depth Dispatch)

Because the maximum depth is $d \leq 3$ and is known at compile time from the `MAX_HYPERTREE_DEPTH` placeholder, the circuit dispatches statically on d rather than using a variable-depth loop:

Listing 1: Depth dispatch in `threshold_aggregate.py`

```

1 @inline
2 def hypertree_merkle_verify(leaf_digest, proof_path, leaf_idx, depth, out_root):
3     match depth:
4         case 1: _hypertree_verify_d1(leaf_digest, proof_path, leaf_idx, out_root)
5         case 2: _hypertree_verify_d2(leaf_digest, proof_path, leaf_idx, out_root)
6         case 3: _hypertree_verify_d3(leaf_digest, proof_path, leaf_idx, out_root)
7     return

```

Each `_hypertree_verify_d` function is a fixed chain of `Poseidon216` calls with a `match` on the bit decomposition of `leaf_idx`. The bit decomposition is *big-endian*: bit index `depth - 1 - j` controls the direction (left/right) at Merkle level j (counted from the leaves). Left child means $b = 0$: `Poseidon2(current, sibling)`; right child means $b = 1$: `Poseidon2(sibling, current)`. See Listing 2 for an example with depth 3.

Listing 2: Depth-3 Merkle verification

```

1 def _hypertree_verify_d3(leaf_digest, proof, leaf_idx, out_root):
2     bits = checked_decompose_bits_small_value_const(leaf_idx, 3) # [MSB, mid, LSB]
3     temp0 = Array(DIGEST_LEN)
4     temp1 = Array(DIGEST_LEN)
5     match bits[2]: # level 0 (leaves): LSB
6         case 0: poseidon16(leaf_digest, proof, temp0)
7         case 1: poseidon16(proof, leaf_digest, temp0)
8     match bits[1]: # level 1
9         case 0: poseidon16(temp0, proof + DIGEST_LEN, temp1)
10        case 1: poseidon16(proof + DIGEST_LEN, temp0, temp1)
11    match bits[0]: # level 2 (root's children): MSB
12        case 0: poseidon16(temp1, proof + 2*DIGEST_LEN, out_root)
13        case 1: poseidon16(proof + 2*DIGEST_LEN, temp1, out_root)
14    return

```

4.2.5 Root Assertion Pattern

A subtle correctness point: the signer root r_{ℓ_i} passed into `xmss_verify` is a *hint* in the private input at a known address. The XMSS circuit writes the re-derived root to that address via the memory-write assertion. The hypertree circuit then reads that same address to begin the Merkle path computation.

The hypertree root $\hat{r}_G$ is computed into a *fresh* buffer (not into the `all_pubkeys` region), because that region may already have been written. An explicit element-wise assertion

$$\hat{r}_G[j] = r_G[j], \quad j = 0, \dots, 7$$

is emitted rather than relying on the write-once memory assertion pattern. This design choice makes correctness independent of the memory state of `all_pubkeys`.

4.3 Poseidon Precomputation

The `leanProver` requires all `Poseidon216` input–output pairs to be provided before trace generation (the `Poseidon216` table is populated from hints, not re-evaluated in the execution trace). For threshold signatures, each signer contributes:

- All `Poseidon2` calls from WOTS chain evaluation and XMSS Merkle path.
- $d \leq 3$ additional `Poseidon2` calls for the hypertree path.

The function `threshold_verify_with_poseidon_trace` in `crates/xmss/src/hypertree.rs` performs native verification while collecting the full trace:

Listing 3: Trace collection for hypertree path (Rust, `hypertree.rs`)

```
1 for sibling in &proof {
2     let is_left = (idx & 1) == 0;
3     current = if is_left {
4         poseidon16_compress_with_trace(&current, sibling, &mut poseidon_trace)
5     } else {
6         poseidon16_compress_with_trace(sibling, &current, &mut poseidon_trace)
7     };
8     idx >>= 1;
9 }
```

All threshold `Poseidon2` traces are parallelised over groups with Rayon, then merged with the raw XMSS traces before being passed to `prove_execution`.

5 Implications for Distributed Validators

5.1 Key Registration

In the pre-quantum DV model, an operator cluster registers a single BLS key on the Ethereum beacon chain. Under `leanConsensus`, the registration changes: the cluster registers the *hypertree root* r_G along with the threshold parameters (k, n) . This root is a 248-bit hash-based commitment to all n members’ XMSS public keys. The hypertree is constructed off-chain by any cluster member (or a trusted coordinator) as:

1. Each node independently generates an XMSS key pair (sk_i, pk_i) with $pk_i = r_i \in \mathbb{F}_p^8$.
2. All n roots $r_1, \dots, r_n$ are shared to all nodes that are part of the DV cluster.
3. Any member computes $\mathcal{T}$ and the joint key $r_G = \mathcal{T}.\text{Root}$.
4. r_G is submitted for registration as the joint public key.

No trusted dealer is required, and no secret is shared between operators: each holds only their own XMSS key.

5.2 Signing Protocol

During operation, the signing protocol for a slot s proceeds as:

1. The consensus layer broadcasts the message m (e.g., an attestation beacon block root) and slot s .
2. Each of the n reachable operators independently signs: $\sigma_i \leftarrow \text{XMSS.Sign}(sk_i, m, s)$.
3. A designated *aggregator* (possibly rotating) collects at least k valid signatures.

4. The aggregator constructs $\Sigma = (\mathbf{i}, \sigma)$ and calls `aggregate([], [], [(G, sigma)], m, s)`.
5. The resulting `AggregatedSigs` is submitted to the Ethereum consensus layer (or to a parent aggregation node).

The XMSS signing operations are entirely independent and can proceed in parallel. The only sequential step is proof generation, which takes $O(k)$ XMSS verification costs in the circuit.

6 Open Challenges for Distributed Validator Deployment

6.1 Cluster Mutability and Key Rotation

The problem. The hypertree root r_G is a static commitment to n specific XMSS public keys. Once r_G is registered on-chain, it cannot be updated without a new on-chain transaction. Real DV clusters need to handle:

- *Member churn*: an operator leaves and is replaced.
- *Key rotation*: an operator suspects key compromise and generates a fresh XMSS key pair.
- *Threshold adjustment*: the cluster policy changes (e.g., from 3-of-5 to 4-of-5 after detecting an unreliable member).

Current limitation. The current implementation has no on-chain mechanism for updating r_G . A cluster that needs to change membership must:

1. Generate a new hypertree $\mathcal{T}'$ with updated members.
2. Submit an exit transaction for the old validator (identified by r_G).
3. Register a new validator with the new joint key r'_G .

This process requires waiting for the exit queue and incurs a new deposit.

References

- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. Whir: Reed–solomon proximity testing with super-fast verification. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 214–243, Cham, 2025. Springer Nature Switzerland.
- [Fou23] Ethereum Foundation. Distributed validator technology. <https://ethereum.org/en/staking/dvt/>, 2023. Accessed: 2025-02-10.
- [GPR21] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo – a Turing-complete STARK-friendly CPU architecture. Cryptology ePrint Archive, Paper 2021/1063, 2021.
- [Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530, 2022.
- [KCLM22] Irakliy Khaburzaniya, Konstantinos Chalkias, Kevin Lewi, and Harjasleen Malvai. Aggregating and Thresholdizing Hash-based Signatures using STARKs. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, ASIA CCS ’22, page 393–407, New York, NY, USA, 2022. Association for Computing Machinery.

- [KLL25] John Kelsey, Nathalie Lang, and Stefan Lucks. Turning Hash-Based Signatures into Distributed Signatures and Threshold Signatures. *IACR Communications in Cryptology*, 2(2), 2025.
- [Set20] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020*, pages 704–737, Cham, 2020. Springer International Publishing.